MLDS 490 Advanced Algorithms for Artificial Intelligence

Assignment #2

Yuxuan Liang

GitHub username: `yuxuanliang1`

Repository name: `MLDS490-Assignment2`

Repository URL: `https://github.com/yuxuanliang1/MLDS490-Assignment2`

November 11, 2026

Abstract

This report evaluates original PyTorch implementations of Classic Deep Q-Networks in CartPole-v1 and MsPacman-v5. Both agents use randomized experience replay, independent target networks, Huber loss, Adam, and gradient clipping. CartPole uses a two-hidden-layer multilayer perceptron and discount factor 0.95; MsPacman uses a single-frame convolutional network and discount factor 0.99. All neural network training ran sequentially on one RTX 5060 Ti on Northwestern's deepdish5. The selected CartPole policy obtained a mean undiscounted return of 499.266 with population standard deviation 11.732 over exactly 500 greedy episodes; 498 episodes reached the 500-step limit. MsPacman completed 3,114 training games before its decision budget expired and obtained a mean score of 1,501.06 with population standard deviation 493.721 over a separate 500-game evaluation. Controlled target-network, replay, discount, exploration, and seed experiments expose substantial sensitivity, including finite Q-value inflation. Independent Bellman diagnostics, altered initial states, and checkpoint-recovery experiments test specific correctness and reliability claims. Action maps and image-conditioned Q preferences complement reward curves. These measurements support learning, while limited full-budget replication, reused validation panels, and remaining failures preclude claims of universal convergence or robustness.

1 Introduction

Reinforcement learning estimates the consequences of actions from interaction. Deep Q-Networks (DQN) approximate action values with a neural network, select actions from those estimates, and improve them using bootstrapped targets [4]. Experience replay reuses transitions in randomly sampled batches; a separate target network slows changes in the regression targets. These are the stabilization mechanisms emphasized in the Week 6 lecture and lab [2, 5].

The two assignment environments exercise different aspects of this method. CartPole-v1 exposes cart position, cart velocity, pole angle, and angular velocity. Two actions apply horizontal force, and the goal is to balance for all 500 steps. MsPacman-v5 instead supplies RGB images and nine discrete commands. Visual compression, hidden motion, irregular rewards, and longer credit assignment make its value estimation and exploration substantially harder. Its raw game score and CartPole's unit-reward return are different quantities and are not compared numerically as a common measure of difficulty.

The objectives are to implement inspectable Classic DQN, measure learned greedy policies on exactly 500 episodes per game, and test whether the algorithm behaves as expected under controlled interventions. The assignment's required discount factors, 0.95 and 0.99, remain the primary settings. Validation includes negative and inconclusive results rather than treating passing software tests as evidence of successful learning. This report is assembled from existing artifacts; no additional training or GPU experiments were conducted during report preparation.

2 DQN methodology

For a policy π, the action-value function is

$$Q^{\pi}(s,a) = \mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s, a_0 = a\right]. \tag{1}$$

For a Markov state, the optimal value satisfies the Bellman relation

$$Q^{*}(s,a) = \mathbb{E}\left[r + \gamma(1-d)\max_{a'} Q^{*}(s',a') \mid s,a\right], \tag{2}$$

where d indicates genuine termination. In the visual experiment a single image is used as a state representation; it need not be Markov because motion is hidden. This is a representational limitation of applying the same update there.

The online network Q_θ outputs one real-valued estimate per action. Its outputs are discounted returns, with no softmax or probability normalization. Training chooses a uniformly random action with probability ϵ and otherwise $\arg\max_a Q_\theta(s,a)$. With n global interaction decisions,

$$\epsilon(n) = \epsilon_0 + (\epsilon_f - \epsilon_0)\min(n/N_\epsilon, 1). \tag{3}$$

Evaluation fixes $\epsilon = 0$; environment stochasticity, including Atari sticky actions, remains active.

The replay ring stores $(s, a, r, s', \texttt{terminated}, \texttt{truncated})$ in preallocated NumPy arrays. It overwrites the oldest data at capacity. Baseline minibatches are uniformly sampled without replacement within each batch and resampled at each update. This reduces temporal correlation but does not make successive overlapping batches statistically independent. Updates start only after the configured warmup.

The target network Q_{θ^-} starts as a copy of the online network, has gradients disabled, and supplies the detached target

$$y_i = r_i + \gamma(1-d_i)\max_{a'} Q_{\theta^-}(s_i', a'). \tag{4}$$

This is Classic DQN: action maximization and evaluation both use the target network, rather than the Double DQN alternative. With residual $\delta_i = Q_\theta(s_i,a_i) - y_i$, the minimized minibatch loss is

$$L(\theta) = \frac{1}{B}\sum_{i=1}^{B} h(\delta_i), \qquad h(\delta) = \begin{cases} \frac{1}{2}\delta^2, & |\delta| \le 1, \\ |\delta| - \frac{1}{2}, & |\delta| > 1. \end{cases} \tag{5}$$

Adam updates only online parameters; the gradient vector is clipped to norm 10. Logged gradient norms are measured *before* clipping and can exceed 10. Hard synchronization copies online weights after each specified number of optimizer updates. Between copies, lagged targets limit the feedback from changing online predictions into their own regression labels. This reduces one source of instability, without guaranteeing convergence under nonlinear approximation. NaN/Inf checks cover Q-values, targets, losses, gradients, and updated weights. Finite but grossly inflated estimates still require scientific diagnostics.

2.1 Termination, truncation, and reward accounting

Gymnasium separates genuine MDP termination from external time limits [7]. Interaction stops on either flag; no further step is issued until reset. Both flags are retained, and the final transition's reward is counted. Only `terminated` masks Equation (4). For example, with $r = 1$, $\gamma = 0.95$, and next maximum Q-value 4, the target is 4.8 for an ordinary or time-limited transition, but 1 for genuine failure. Masking `terminated OR truncated` would incorrectly model a time cutoff as an absorbing state. If both flags occur, termination masks the continuation. This is a continuing-task bootstrap convention; it does not encode a separate finite-horizon value function with remaining time in the observation.

2.2 Training and model selection protocol

Each iteration executes an epsilon-greedy action, records the complete transition, inserts it into replay, optionally samples a minibatch and applies Equations (4)–(5), and then synchronizes the target when due. Episode summaries include return, length, epsilon, loss, gradient norm, Q statistics, decisions, updates, and elapsed time. Separate environments provide periodic greedy validation. A strictly improved validation mean saves an immutable best checkpoint; earliest checkpoints win ties. Final snapshots are also retained. Episode-boundary budget checks permit small overshoots. Neither experiment used a convergence-based stopping rule.

Training rewards describe exploratory behavior under changing weights; validation means select snapshots; the 500-episode panels assess frozen selected policies. All reported within-model standard deviations use the population convention $\mathrm{SD}_0 = \sqrt{\sum_i (R_i - \bar{R})^2 / n}$. Between-training-run variation is identified separately and is not the standard deviation of individual evaluation episodes.

3 CartPole-v1

3.1 Implementation and training configuration

The state $(x, \dot{x}, \theta, \dot{\theta})$ is used directly as float32. The MLP has architecture $4 \to 128 \to 128 \to 2$, with ReLU after each hidden layer and linear action-value outputs. Action 0 pushes left and action 1 pushes right. Table 1 reports the selected run's saved configuration rather than assuming that current CLI defaults reproduce its budget.

Seed 33 completed 942 episodes, 80,014 decisions, 79,015 optimizer updates, and 158 target copies in 180.099 seconds. Its best validation mean was 500.0 at episode 942, selecting `best_ep0942.pt`. Validation uses reset seeds 50,000–50,009. Among baseline training seeds 11, 22, and 33, this model had the highest validation mean. The smaller seed-11 and seed-22 validation maxima were 407.6 and 473.7. All three seeds' test results are disclosed in Section 5.1.

3.2 Learning curves and the Q-value scale

For episode e, the Q statistic is $\max_a Q_{\theta_e^{\mathrm{end}}}(s_{e,0}, a)$: the episode's initial observation is evaluated using the online weights *after* that episode's updates. Figure 1 therefore mixes changing weights with independently changing reset states; it is not a maximum over the trajectory or replay buffer.

Table 1: CartPole primary configuration (training seed 33).

Setting	Value
Environment; discount	CartPole-v1; $\gamma = 0.95$
Optimizer; learning rate	Adam; 5×10^{-4}
Replay capacity; warmup	50,000; 1,000 transitions
Batch size; update frequency	128; every decision after warmup
Target synchronization	Hard copy every 500 optimizer updates
Loss; gradient norm cap	Huber ($\beta = 1$); 10
Exploration	$1.0 \rightarrow 0.05$ linearly over 20,000 decisions
Validation	Every 25 episodes and final episode; 10 greedy resets
Limits	1,000 episodes or 80,000 decisions; native 500-step episodes
Stopping	Budget at episode boundary; early stopping disabled

Its observed range is 0.0392–20.8084, with final value 20.6677. Continuing unit rewards have scale

$$\sum_{t=0}^{\infty} 0.95^t = \frac{1}{1 - 0.95} = 20, \tag{6}$$

and the 500-step discounted sum is effectively 20 as well. Thus undiscounted reward 500 does not imply Q-values near 500. Slight overshoot of 20 is possible under approximation; the network does not enforce this theoretical reference.

Figure 2 shows all exploratory returns and the strict trailing 100-episode mean, first defined at episode 100. The first 100 episodes average 22.83; the final 100 average 177.32. At the final exploration floor, random pushes can still cause early failure, and weights vary throughout the averaging window. The separate greedy validation curve fluctuates substantially before reaching its highest score near the budget boundary. Neither an isolated successful training episode nor the final validation maximum establishes sustained convergence.

3.3 Exactly 500 greedy evaluation episodes

The selected checkpoint was evaluated with epsilon zero on distinct reset seeds 100,000–100,499. Table 2 and Figure 3 report the complete panel. Exactly 498/500 episodes reached 500 steps; the remaining returns were 346 and 287 at seeds 100,211 and 100,261. The high mean therefore coexists with two genuine failures. A different final-snapshot diagnostic used 100 other seeds and is not substituted for this evaluation.

3.4 Non-standard visualization: a control-policy slice

Figure 4 evaluates both action values on a 161×161 grid of pole angle and angular velocity, with $x = \dot{x} = 0$. Decision regions use the argmax, and contours show $Q(s, \text{right}) - Q(s, \text{left})$; the zero contour marks equal preferences. The saved grid is `metrics/policy_grid.csv` within the seed-33 run.

The map is asymmetric. Near $\theta = \pm 0.05$ rad and zero angular velocity, both sampled states select right; the probes at zero angle and angular velocities ± 0.5 rad/s also select right. These do not follow a symmetric local corrective rule. Larger paired probes near $(-0.10, -1)$ and $(0.10, 1)$ select left and

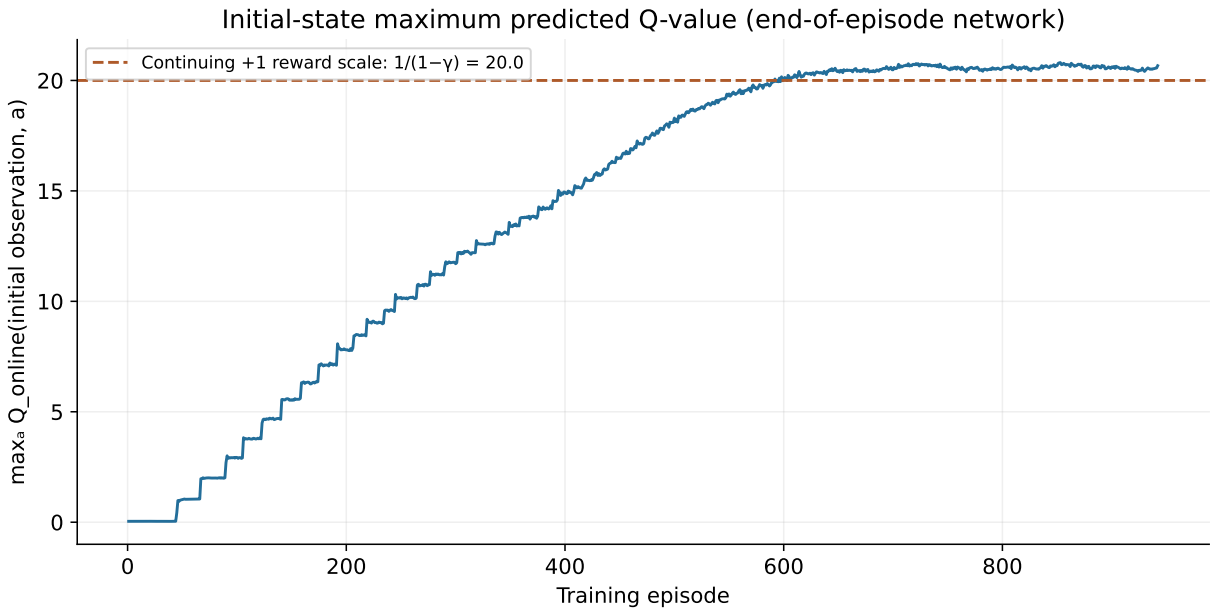


Figure 1: CartPole maximum action value at each episode's initial observation, using the online network after that episode's updates. The discounted unit-reward reference is 20, not the undiscounted episode limit of 500.

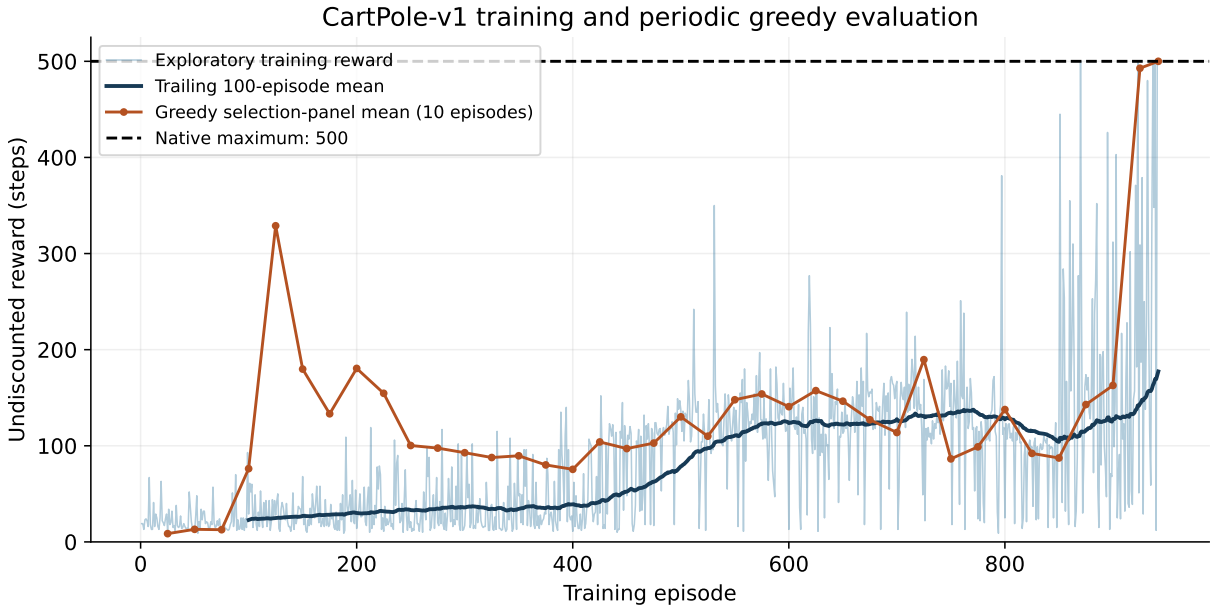


Figure 2: CartPole seed-33 exploratory training returns, trailing 100-episode mean, and separate greedy validation means. The reward-500 line marks the native time limit. Validation episodes are excluded from the moving average.

Table 2: Frozen selected policies: exactly 500 greedy episodes per environment. Standard deviations describe individual returns within each model, with $ddof = 0$. The games' reward units differ.

Statistic	CartPole (unit rewards)	MsPacman (raw score)
Training seed; selected episode	33; 942	41; 3,114
Mean	499.266	1,501.060
Population standard deviation	11.731634	493.720849
Median	500	1,430
Minimum / maximum	287 / 500	590 / 4,800
True terminations / truncations	2 / 498	500 / 0
Perfect 500-step episodes	498/500 (99.6%)	Not applicable

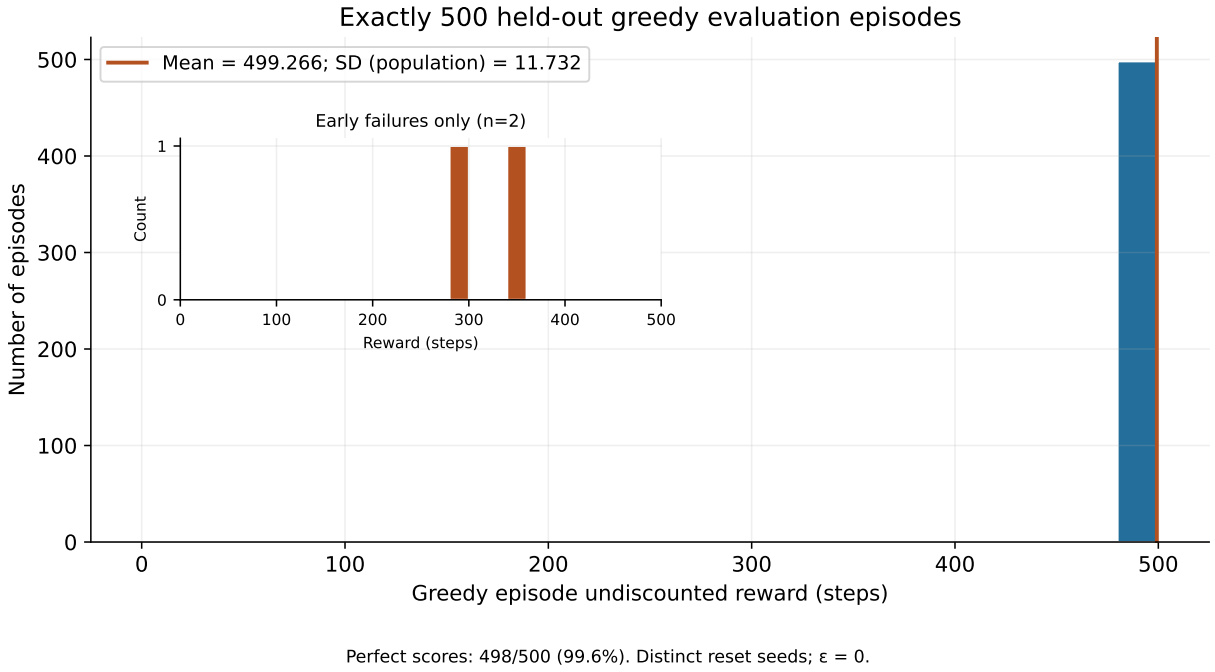


Figure 3: CartPole selected-checkpoint reward histogram for exactly 500 greedy episodes. Mean 499.266, population SD 11.731634, and 498 perfect episodes. The two low-return episodes are retained in all statistics.

right, respectively, consistent with moving the cart beneath the falling pole. These observations do not establish optimality: cart position and velocity are suppressed, and closed-loop trajectories determine actual stability. Agreement with the heuristic "push right if $\theta + 0.2\dot{\theta} > 0$" is only 49.97% in the central slice. That heuristic is an interpretive comparator, not a correctness oracle.

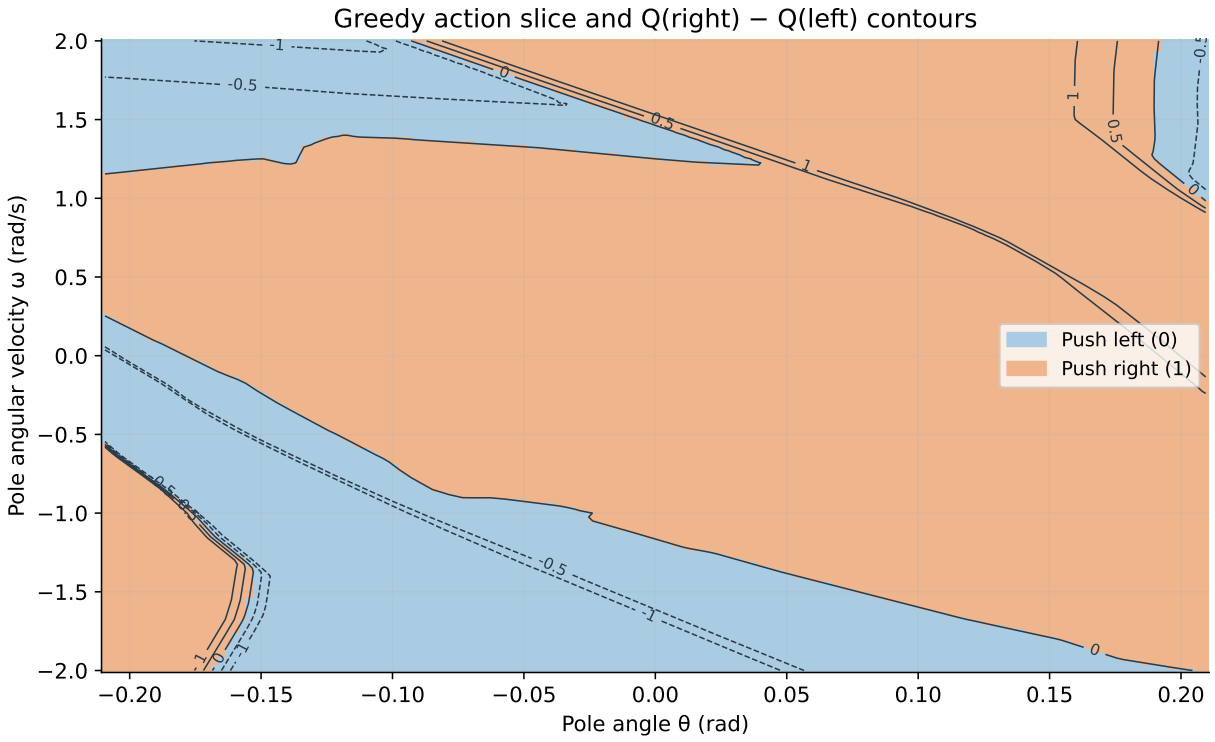


Figure 4: Greedy CartPole action regions and action-value difference contours on the fixed slice $x = \dot{x} = 0$. Angle is in radians; angular velocity is in rad/s. This is a two-dimensional slice of a four-dimensional policy, not a complete recovery-region map.

4 MsPacman-v5

4.1 Environment, representation, and architecture

The installed ALE registers `ALE/MsPacman-v5`. The implementation creates the assignment's literal `MsPacman-v5` ID as an alias with identical entry point and configuration, rejecting conflicting registrations. A recorded test compares RGB observations, rewards, and end flags for 120 common actions under the alias and canonical environment. This changes the registration name, not the game dynamics. ALE is the emulator interface [1]; explicit evaluation protocols matter, particularly sticky actions [3].

The environment supplies $210 \times 160 \times 3$ uint8 RGB images and nine commands: NOOP, UP, RIGHT, LEFT, DOWN, UPRIGHT, UPLEFT, DOWNRIGHT, DOWNLEFT (indices 0–8). Native frameskip is four, sticky-action probability is 0.25, and the native limit is 108,000 raw frames per game. There is no additional action-repeat wrapper, random no-op reset, frame stack, reward clipping, or episodic-life termination. Life loss continues the same game. Headless execution does not request a display. Discounting applies per agent decision, not per raw emulator frame.

Preprocessing follows the assignment exactly. The slice `obs[1:176:2, ::2]` crops and downsamples to 88×80 pixels. RGB channels are summed in signed int32 to prevent overflow. Every sum equal to $210 + 164 + 74 = 448$ is set to zero; this is an intensity-sum mask, not a semantic background detector. Integer division by three followed by subtraction of 128 produces int8 intensities. The HWC array is transposed to shape $(1, 88, 80)$, and the CNN converts it to float32 and divides by 128. The representable input range is $[-1, 127/128]$. This differs from a conventional stacked-frame Atari pipeline.

The CNN uses three unpadded convolutions: $1 \to 32$ channels with kernel 8 and stride 4; $32 \to 64$ with kernel 4 and stride 2; and $64 \to 64$ with kernel 3 and stride 1. ReLU follows each. The resulting $64 \times 7 \times 6 = 2,688$ features feed a 512-unit ReLU layer and nine linear Q outputs. There are 1,453,225 parameters. Replay stores current and next int8 images separately. Its preallocated arrays occupy 704,700,000 bytes (0.656 GiB), avoiding float32 image storage; a RAM guard rejects oversized allocation requests.

Table 3: Saved MsPacman primary configuration, seed 41. A decision is an agent action, advancing up to four raw frames; optimizer cadence is a separate quantity.

Setting	Value
Environment; discount	MsPacman-v5; $\gamma = 0.99$
Optimizer; learning rate	Adam; 10^{-4}
Replay capacity; warmup	50,000; 10,000 decisions
Batch size; update frequency	32; every four decisions after warmup
Target synchronization	Hard copy every 5,000 optimizer updates
Loss; gradient norm cap	Huber ($\beta = 1$); 10
Exploration	$1.0 \to 0.1$ linearly over 500,000 decisions
Validation	Every 100 games and at end; 10 fixed greedy resets
Planned limits	4,000 games, 2,000,000 decisions, or 4 hours
Recovery cadence	Every 250 completed games and at final boundary

4.2 Training exposure and learning evidence

Training completed 3,114 games, 2,000,528 agent decisions, and 7,992,770 measured raw frames. It performed 497,633 optimizer updates and 99 hard target copies. Elapsed training time was 3,268.544 seconds (54.48 minutes), including periodic validation and recovery writes. Peak allocated CUDA tensors occupied 103.04 MiB; final process RSS was 3.308 GiB. CUDA context and reserved memory are additional to allocated tensor memory. The decision cap was checked at a game boundary and overshot by 528 decisions. Training did not complete the assignment's suggested 4,000–6,000-game scale and did not stop on a convergence claim.

Figure 5 uses a fixed panel of 32 unique images collected before training by random play at seed 410,000, every 25 decisions. After each training game it plots the *mean over images of the maximum over actions*. The band is population SD across these fixed images, not a confidence interval over runs. The mean statistic ranged from 0.026125 to 85.196686 and finished at 84.012657. Its increase shows changing value estimates; it does not establish calibration. Raw rewards are not constant unit rewards, so $1/(1-0.99) = 100$ is not a bound on MsPacman Q-values or its undiscounted game score.

Figure 6 supplies direct performance evidence. First-100 and last-100 exploratory means were 272.1 and 994.2. Greedy validation uses seeds 600,000–600,009: the untrained model scored 210 on average, the episode-100 model 110, the episode-1,600 model 707, and the selected episode-3,114 model 1,387. Temporary regressions remain visible, including means 1,205 at episode 2,800, 811 at 2,900, and 830 at 3,000. The overall improvement is substantial, but neither the training curve nor the reused ten-seed validation panel is monotonic. Uniform random actions on that panel scored 252 (population SD 77.563).

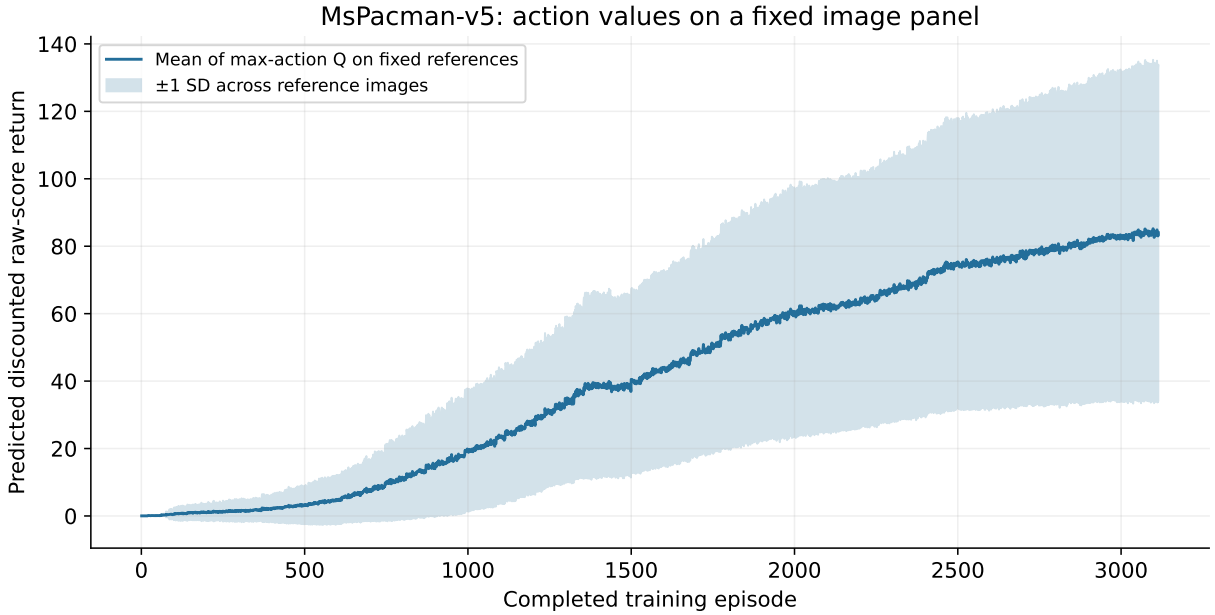


Figure 5: MsPacman maximum Q-values on a fixed 32-image panel, averaged over images after each completed training game. Shading is SD across images, not between training seeds. Images come from early random play.

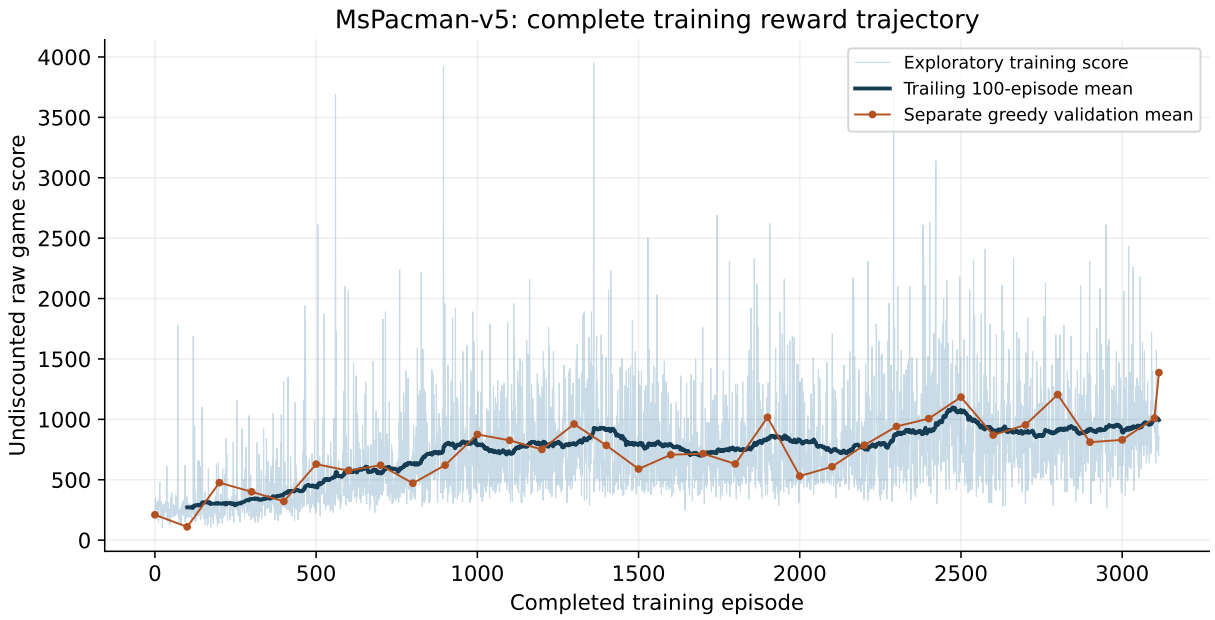


Figure 6: All MsPacman exploratory scores, strict trailing 100-game mean, and separate greedy validation means. Rewards are raw and include every life. Performance regressions qualify the overall upward learning trend.

4.3 Selected policy and 500-game evaluation

The highest ten-seed validation mean selected `best_ep03114.pt` at the last completed game; the final model was separately saved. Final evaluation used epsilon zero with seeds 900,000–900,499, disjoint from selection and short-run diagnostics. Exactly 500 games completed in 251.658 seconds. Mean score was 1,501.06, population SD 493.720849, median 1,430, and range 590–4,800 (Table 2; Figure 7). All ended by genuine game-over, with no frame-limit truncations. Mean length was 856.52 decisions or 3,423.08 raw frames. Native repeats can end early on game-over, so measured raw frames need not equal exactly four times decisions. The spread shows considerable conditional performance variation across resets; these games do not measure variability between independently trained full-budget models.

4.4 Non-standard visualization: image-conditioned action preferences

Figure 8 displays six of the fixed reference images and all nine saved action values. Across the complete 32-image panel, seven different commands are greedy, 93.75% of greedy commands differ from initialization, and the median top-two Q margin is 0.291349. In the displayed examples, reference 0 prefers NOOP, 6 prefers UP, 18 prefers UPLEFT, and 31 prefers LEFT. Preferences therefore vary with the observed image; a constant-action policy does not explain this panel. Small margins and differing absolute scales also show why a Q margin is neither a calibrated confidence nor a probability of an optimal action.

The panel represents early random-play images, not all later-game situations. Some similar-looking scenes choose NOOP, and the single-frame input hides motion. State-dependent outputs alone

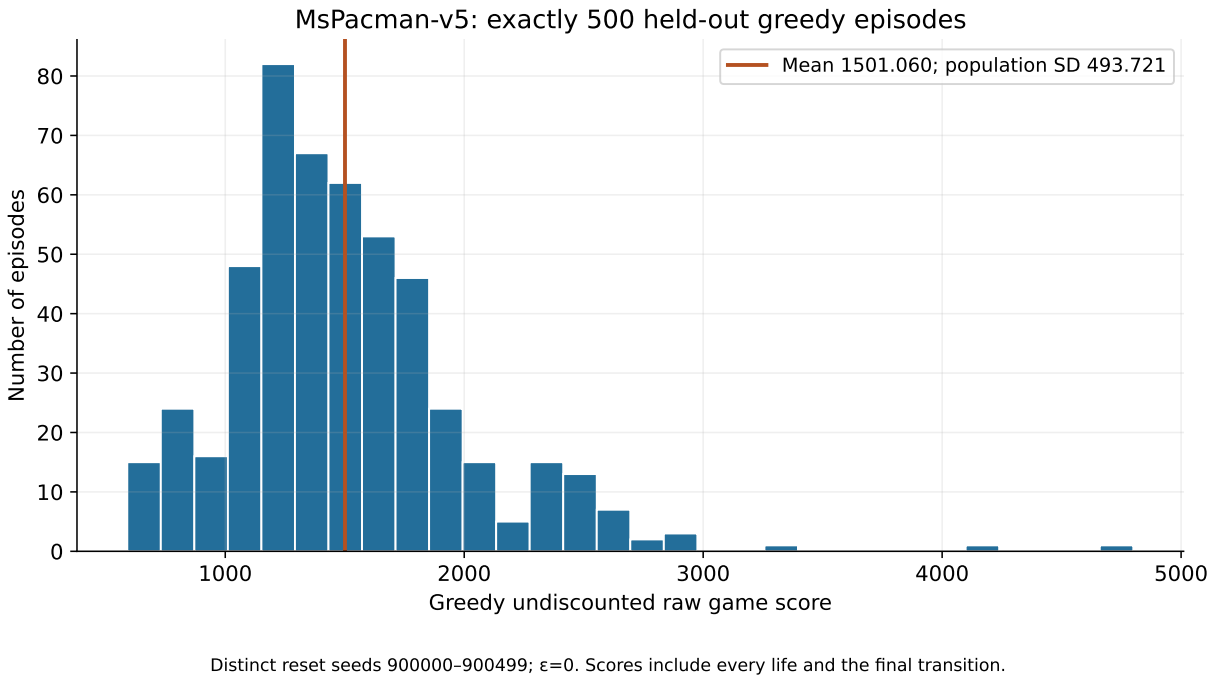


Figure 7: Exactly 500 held-out MsPacman games from the selected greedy checkpoint. Mean 1,501.06 and population SD 493.720849. Sticky actions remain enabled; all scores include the final transition and all lives.

cannot establish collision avoidance, effective navigation, or optimal play. The full RGB thumbnails provide human context; the CNN sees only the specified preprocessed crop. The report version adjusts layout and labels for letter-paper readability using the same saved values and pixels; the original experiment PDF is retained alongside it.

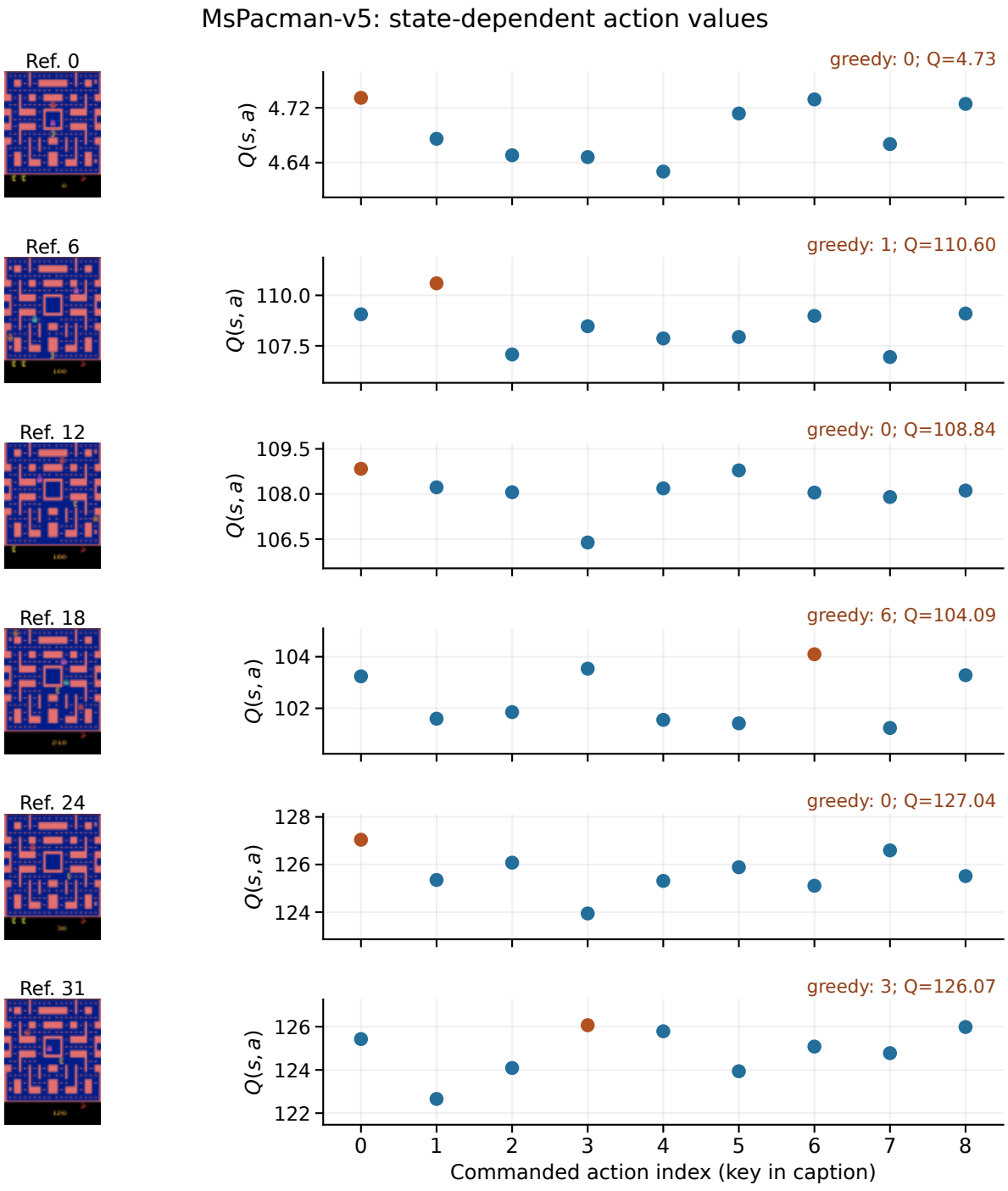


Figure 8: MsPacman image-conditioned Q preferences for references 0, 6, 12, 18, 24, and 31. Each row has its own vertical scale. Orange marks the greedy command; all nine values are plotted. Action indices: 0 NOOP, 1 UP, 2 RIGHT, 3 LEFT, 4 DOWN, 5 UPRIGHT, 6 UPLEFT, 7 DOWNRIGHT, 8 DOWNLEFT. Full RGB is shown only for context. Formatting-only adaptation of saved experiment data; no new inference.

5 Scientific validation and edge cases

The experiments below test hypotheses about learning and numerical behavior, distinct from unit tests. Both games retain the same environment, architecture, optimizer, replay capacity, warmup, batch size, validation panel, and checkpoint selection rule within a controlled comparison, except for the explicitly changed factor. Actual exposure and evaluation sample size are reported: equal nominal caps do not guarantee identical decisions when stopping occurs at episode ends. Poor learning with finite computations is an experimental outcome, not by itself an implementation bug.

5.1 CartPole controlled experiments

Table 4 reports all eight completed runs. Ablations compare against *seed 11*, preserving the training seed, rather than against the stronger selected seed 33. Every selected checkpoint received 500 greedy resets on the common evaluation panel. Limits were 1,000 episodes or 80,000 decisions. The every-update and gamma-0.99 conditions reached the episode ceiling first, so their lower exposure is a material limitation of the comparison. Loss and extreme-Q diagnostics below summarize the entire training run, whereas evaluation returns describe validation-selected snapshots and final-state Q statistics describe the last network.

Table 4: CartPole controlled experiments and independent training seeds. Return mean and SD refer to each validation-selected model on 500 resets. "Perfect" is the percentage reaching 500 steps. Exposure is actual, not the nominal cap.

Condition (training seed)	Episodes	Decisions	Mean	SD_0	Perfect
Hard-500 baseline (11)	894	80,216	407.824	115.897	53.6%
Same configuration (22)	881	80,092	392.584	125.059	52.2%
Same configuration (33)	942	80,014	499.266	11.732	99.6%
Soft, old weight .999 (11)	948	80,132	223.292	62.361	0.0%
Hard every update (11)	1,000	15,769	9.360	0.774	0.0%
Recent-only batches (11)	794	80,183	385.140	138.633	57.8%
$\gamma = 0.90$ (11)	984	80,156	393.332	48.506	3.4%
$\gamma = 0.99$ (11)	1,000	58,648	163.250	58.883	0.0%

Training-seed sensitivity. The hypothesis is that initialization, exploration, reset, and replay randomness can materially alter learning under identical hyperparameters. Only the seed changes across 11, 22, and 33. The three evaluation means average 433.224667; population SD *between run means* is 47.110917 (sample SD 57.698853), and the mean perfect-episode fraction is 68.47%. No run achieved 500/500 perfect episodes. Thus the best run is strong but not representative of reliably learned 500-step control across seeds. Three replications cannot characterize rare failures or establish a stable training success probability.

Target lag: hard, soft, and every-update copies. The hypothesis is that lag limits moving-target feedback. At seed 11, the hard-500 baseline is compared with a soft update after every optimizer step,

$$\theta^- \leftarrow 0.999\theta^- + 0.001\theta, \tag{7}$$

and with hard copies every update. The coefficient 0.999 weights the *old* target, following the lecture convention [2]. Soft and every-update mean returns are lower than hard-500 by 184.532 and 398.464, respectively. The every-update run's largest sampled absolute Q-value is 22,336,222, versus 23.260 for hard-500; mean Huber losses are 416,698.357 and 0.07543. This is severe *finite* value inflation relative to the unit-reward scale 20, accompanied by nearly immediate failures. Gradient clipping and nonfinite guards did not prevent it. These observations are consistent with harmful moving-target feedback in this run, not a proof of its sole cause. The different actual exposure and one training seed limit causal generalization. Soft-.999 also has a different effective lag from hard-500, so this is not an isolated test of smoothness.

Correlated replay. The hypothesis is that highly overlapping batches reduce sample diversity and may impair stability. Only sampling changes: the most recent 128 transitions in chronological order replace random minibatches, with capacity and all other settings fixed at seed 11. This retains memory and is not a no-replay method. Mean return falls by 22.684, but the perfect fraction rises from 53.6% to 57.8%. The recent condition has larger return SD and a lower loss 95th percentile (0.13421 versus 0.23922). The mixed result does not establish uniform replay's universal superiority, and lower regression loss does not imply better control.

Discount sensitivity. Changing only gamma to 0.90 or 0.99 tests the joint effect of planning horizon and value scale. Mean returns are 393.332, 407.824, and 163.250 at gamma 0.90, 0.95, and 0.99; final initial-state Q statistics are 10.046, 20.648, and 79.402, against unit-reward scales 10, 20, and 100. The higher-discount run has mean loss 0.48968 versus 0.07543 at 0.95 and hits the episode ceiling after 58,648 decisions. These are finite-budget outcomes, not an asymptotic ranking of discounts. The required primary gamma remains 0.95.

Altered initial states. The hypothesis is that high standard-reset performance need not imply recovery from larger perturbations. The selected seed-33 model is frozen. A 9×9 grid varies angle in $[-0.18, 0.18]$ rad and angular velocity in $[-1.5, 1.5]$ rad/s, with five nuisance cart resets per grid point (seeds 70,000–70,004). After reset, `env.unwrapped.state` is assigned the specified angle and angular velocity; forces, gravity, and time step are unchanged. All 405 episodes completed: mean 402.474074, population SD 170.273223, and 293/405 perfect episodes (72.35%). The minimum return is 1. This is a distribution-shift stress test, not standard CartPole evaluation. Some extreme states may be physically unrecoverable, and five nuisance resets per cell provide limited precision.

5.2 MsPacman learning, ablations, and short-budget replications

The main learning-evidence comparison holds the ten validation reset seeds fixed and changes training exposure. The selected-minus-untrained mean difference is 1,177 points. Its saved paired-bootstrap interval, [986.0, 1,357.025], reuses the panel that selected the model and is selection-biased; it is exploratory, not a confirmatory improvement interval. The random-policy mean 252 and the intermediate checkpoint mean 707 provide additional context for the final validation mean 1,387. The separate 500-game test reports the selected policy's score distribution.

Target-network hypothesis. Replacing hard-5,000 synchronization with every-update copies tests whether removing lag impairs learning, with all other factors fixed at seed 41 and matched decision caps. Diagnostic mean falls from 379 to 210. The largest sampled absolute Q rises from

Table 5: MsPacman short-budget experiments, each capped at 100,000 decisions and evaluated on 30 diagnostic resets (700,000–700,029). All have gamma 0.99; each model is independently selected using ten validation resets. These are not full-budget replications of the 2,000,528-decision main run.

Changed factor (seed)	Games	Decisions	Mean	SD_0	Final ϵ
Baseline (41)	199	100,031	379.000	146.705	.81994
Training seed (52)	203	100,235	591.000	267.462	.81958
Training seed (63)	197	100,343	335.000	175.760	.81938
Target copy every update (41)	198	100,206	210.000	0.000	.81963
Recent-only batches (41)	197	100,299	394.333	203.858	.81946
Epsilon decay 1M (41)	197	100,341	210.000	0.000	.90969

24.585 to 91.881, and loss 95th percentile from 1.18224 to 1.49171. This finite inflation is much smaller than the CartPole ablation's pathology. The selected model is the initial snapshot because later validation never improves its score; the reported final Q statistic describes a different snapshot.

Replay hypothesis. Recent chronological batches of 32 replace uniform batches, keeping capacity, warmup, update ratio, seed, and all other settings fixed. Mean score changes by +15.333, while loss 95th percentile increases to 1.76304. The saved paired 95% bootstrap interval for the score difference is $[-73.675, 124.667]$, spanning zero. This provides no conclusive direction of effect at the short budget. It does not establish equivalence or demonstrate that replay diversity is unimportant later.

Exploration hypothesis. Doubling the linear decay horizon from 500,000 to 1,000,000 decisions tests whether prolonged exploration changes early learning. Both endpoints remain unchanged. Final epsilons are 0.81994 and 0.90969; diagnostic means are 379 and 210. The slower variant also selects its initial snapshot. Neither run reaches its exploration floor, so this cannot determine the preferable full-training schedule.

Uncertainty and replication scope. The saved paired intervals use common diagnostic reset seeds, 10,000 percentile bootstrap resamples, and variant-minus-baseline scores. Every-update and slow-decay differences are both -169, with intervals $[-221.333, -117.333]$ and $[-222.000, -116.658]$. These intervals are conditional on the selected models, describe reset uncertainty, have no multiple-comparison adjustment, and do not measure uncertainty across training seeds. Independent short seeds 41, 52, and 63 have mean-of-means 435.0 and between-run population SD 111.761651. Only one seed has full-budget training, so full-budget reliability remains unmeasured.

5.3 Trajectory and Bellman diagnostics

For CartPole, the frozen selected model and reset seed 31,415 produce a complete ten-step trace of states, both Q-values, actions, rewards, next states, end flags, and checkpoint-based targets. All 10/10 actions match online argmax. An independent float64 NumPy implementation of the saved target network's linear/ReLU layers recomputes Bellman targets for 40 real sampled transitions, including four true terminals and four truncations. Maximum absolute error is 3.01756×10^{-6}. The analytic example in Section 2.1 separately checks the semantic difference between the flags.

For MsPacman, seed 314,159 supplies ten initial decisions and ten later decisions (101–110),

with all nine Q-values and exact processed state/next-state image arrays. All 20/20 actions match argmax. The later window supplements potentially repetitive startup frames. On 36 real replay transitions, including five terminals and no truncations, independent float64 *scalar arithmetic* on saved target Q-values differs by at most 8.54492×10^{-6}. This is not an independently implemented CNN. Synthetic time-limit cases provide the truncation coverage absent from these real transitions. Both games' diagnostics use saved-checkpoint weights; neither reconstructs historical targets at transition collection time.

Checkpoint inference is also checked across devices: CartPole has zero same-CUDA reload discrepancy and maximum CPU–CUDA discrepancy 5.72205×10^{-6} over 128 probes; MsPacman has maximum discrepancy 0.00898743 over its 20 traced images. Greedy actions agree in both cases. These finite probe sets support inference consistency, not bitwise equality across all hardware or states.

5.4 Deterministic repetition and interruption recovery

Repeating MsPacman seed 41 under the same GPU/software matched the main run's first 100,031 recorded transitions, including scalar fields and image checksums. Online and target parameters matched exactly at episode 100, the last common *saved* checkpoint; no claim is made about unsaved final-prefix weights. Checksums can collide, so matching checksums alone are not proof of pixel identity. This is a repeated prefix, not a second complete main experiment.

Full Atari recovery saves online and target weights, Adam state, update counters, global epsilon step, replay arrays and ring position, sampling and behavior RNGs, Python/NumPy/PyTorch/CUDA RNGs, and reference/selection metadata. Restoration occurs at complete-game boundaries; the next reset seed follows the episode index. The run's reference files and retained best models must accompany the recovery checkpoint and replay sidecar. Model-only snapshots do not provide full recovery.

A separate four-game CUDA smoke compared uninterrupted execution, a two-plus-two resume, and rollback/re-execution from game two. It used target interval 50 *for the smoke only*, completing 471 optimizer updates and nine target copies. Both resume and rollback maximum online-parameter differences were zero. Current episode rows were uniquely 1–4; rows beyond the restored boundary were archived before rollback, and resumed checkpoints received distinct names. This tests state restoration and log integrity under a controlled interruption, not long-run Atari convergence. Abrupt loss can still discard the in-flight game and work since the latest recovery boundary.

5.5 Environment-specific edge cases and software evidence

Table 6 connects failure modes to measured or focused-test evidence. The saved focused suites contain 18 passing CartPole tests and 12 passing MsPacman tests. These counts are historical test-log results; no experiments or tests were rerun for this report. They establish specific properties, while the learning experiments establish the broader empirical behavior.

Atari image checksums were unchanged on 2.411% of main decisions; this is an instrumented checksum statistic rather than proof of identical images. Real main/evaluation games did not hit the native frame limit, so truncation evidence comes from explicitly labeled reduced-cap tests. Preflight JSON serialization and launcher argument failures were corrected before main training and remain logged; they are implementation/debugging events, not unsuccessful algorithm conditions silently removed from the experiment tables.

Table 6: Edge-case handling and its evidence. Analytic controllers and shortened frame caps are bookkeeping tests, never replacements for learned-policy evaluation.

Potential failure	Implemented handling	Existing validation evidence
CartPole early failure or final reward omitted	End on either flag; count every reward	One-step failure returns 1; analytic controller reaches 500 with truncation; selected DQN test has 498 such episodes
Time cutoff treated as absorbing	Store both flags; only termination masks	Known targets 4.8 vs 1; four real CartPole truncated samples; native Atari 12-frame and wrapper nine-decision tests
Replay underflow or stale overwritten data	Reject undersized sample; fixed ring	Both suites test wraparound and retained newest data; uniform vs recent learning experiments use the same capacity
Wrong action indexing, exploration, or target gradients	Gather chosen Q; independent frozen target	Known Huber/target cases, epsilon 0/1, global-step schedules, and exact target-sync tests
Atari life loss ends game prematurely	Continue until genuine game-over	Main log: 3,114 game-overs, 6,228 nonterminal life losses, zero truncations
Atari sparse rewards mistaken for a stall	Preserve zero rewards and keep stepping	91.199% zero-reward decisions; longest zero-reward interval 535 decisions; raw rewards span 0–1,610
Pixel overflow or replay memory excess	Int32 sum, int8 storage, RAM guard	Exact black/white pixel arithmetic, shapes and signed range tested; 0.656-GiB replay allocation documented
Numerical divergence hidden by clipping	Stop on nonfinite values; retain finite diagnostics	Nonfinite injection tests and finite black/white Q probes; target ablations expose finite inflation despite clipping
Evaluation contaminates training	Separate environments and epsilon zero	Isolation and reload tests; independent reset panels; alias-equivalence test and headless execution
Interruption duplicates logs or loses state	Full boundary recovery and archival rollback	Four-game resume/rollback equality with nine target copies; matching seed-41 prefix at saved stage

6 Discussion and further experiments

The CartPole best run approaches the 500-step goal on its tested reset distribution, but its two failures, seed sensitivity, right-biased policy slice, and lower stress-test success qualify any robustness claim. The target ablation provides unusually clear evidence that finite value estimates can become scientifically implausible while software guards continue to pass. The replay ablation is less decisive: return mean and perfect-score rate rank variants differently. These results reinforce the lecture's caution that loss reduction and occasional high returns are insufficient measures of successful control.

MsPacman exhibits improved raw scores relative to initialization, random actions, and early checkpoints, together with state-dependent action preferences. Its computational cost is greater, but no CPU/GPU speed comparison was measured. Single-frame observations hide velocity and temporal context; the assignment's crop and intensity mask discard information. Sticky actions, sparse rewards, bootstrap approximation, and a reused small validation panel complicate inference. The decision-limited run ends below the suggested game count, only one full-budget seed is available, and short ablations remain highly exploratory. Neither rising Q-values nor the best validation score proves calibrated values, global optimality, or mathematical convergence.

6.1 Selection and evidence limitations

Within each run, checkpoints were selected by validation rather than the final test. CartPole's final primary designation across three seeds also reads only validation scores. However, per-run test summaries had already been inspected before that final designation. Reporting every seed and avoiding subsequent hyperparameter tuning reduces selective presentation, but the selected best-run test should not be described as a fully preregistered, untouched across-seed confirmatory study. MsPacman's final 500-game panel is separate from checkpoint selection and short-run diagnostics. In both games, repeated use of ten validation seeds can favor a noisy maximum. Additional fresh panels and more independent training runs would strengthen generalization claims.

6.2 What would be tested with an additional three months?

Month 1: reliability and replication. Preregister matched decision/frame budgets, model selection, and a final untouched panel. Prioritize 10 full-budget seeds per game, executed sequentially within a declared compute allocation, before expanding the hyperparameter search. Extend MsPacman beyond its present cap and evaluate learning at common exposure points. Report distributions of seed-level means, CartPole success probability, score quantiles, sample efficiency, wall time, and between-run uncertainty. Sweep learning rate, exploration horizon, and target lag one factor at a time. Match effective lag when comparing hard and soft updates, and do not let an episode ceiling give weak policies much less interaction.

Month 2: controlled algorithm extensions. Keep Classic DQN as the reference and separately test Double DQN [8], dueling heads [9], prioritized replay [6], and multi-step returns. Ask whether they improve reward per decision, reduce finite value inflation, or alter seed variability under equal experience and comparable update budgets. Compare Q estimates with independently estimated discounted rollout returns at fixed states to assess calibration, not just Q magnitude. Factorial follow-ups would test interactions only after the individual interventions are understood.

Month 3: generalization, representation, and robustness. Compare single-frame and four-frame inputs while accounting for increased replay memory. Stratify Atari diagnostics by later levels, life loss, sparse-reward intervals, and ghost encounters. Expand CartPole's recovery map across all four state dimensions and compare failures with physically recoverable regions. Test observation noise, delay, and explicitly labeled dynamics shifts; report return quantiles, recovery rates, and action consistency. Replicate recovery across devices and software versions and add a small number of additional Atari benchmarks only after within-game reliability is measured. These are proposed studies, not completed results of this assignment.

7 Reproducibility and source availability

GitHub username: [GitHub username]. **Repository name:** [Repository name].
Public source link: [Public repository URL required]. These author-supplied metadata fields must be completed before submission; no repository URL is inferred from machine account names.

7.1 Execution environment and retained evidence

All substantive neural-network training ran on `mlds-deepdish5.ads.northwestern.edu`, sequentially using one NVIDIA GeForce RTX 5060 Ti (nominal 16 GB), compute capability 12.0. The recorded physical UUID is `GPU-cef65388-88b0-f39a-6bac-c5316eb78540`; the selected process sees only logical `cuda:0`. UUID selection avoids a documented NVML/CUDA ordinal mismatch. The observed build supports `sm_120`; a real CUDA matrix product returned the expected sum 3,680. Training records place inputs, Q-values, targets, losses, gradients, and Adam moment tensors on CUDA. Environment stepping and replay storage remain on CPU. No other user's process was terminated.

The saved software environment is Python 3.12.6, PyTorch 2.14.0 with CUDA 13.0, driver 615.71.09, Gymnasium 1.4.0, ALE 0.10.2, NumPy 2.5.3, Matplotlib 3.11.2, pandas 3.0.5, and pytest 9.2.0. A project environment reused compatible installed packages without modifying shared installations. Python, NumPy, PyTorch, CUDA, replay, behavior, action-space, and environment reset seeds are controlled; deterministic kernels are requested. Exact repetition across versions or hardware is not guaranteed. CartPole checkpoints support inference reload but do not contain full optimizer/replay/RNG recovery; MsPacman's separate recovery checkpoints do.

Each run retains `metrics/config.json`, selection/training summaries, episode/update/transition logs, model snapshots, and evaluation CSVs. Raw artifacts were cross-checked against comparison tables, notes, and both selected checkpoints' CPU-loaded metadata. Report preparation verified all eight CartPole and six short MsPacman evaluation tables and both complete 500-episode panels. The report's `audit/` directory records numerical checks, source hashes, figure provenance, and discrepancies. Historical CartPole notes stating that MsPacman is unimplemented describe the earlier Part-(a) stage; actual Part-(b) artifacts supersede that scope statement. Saved run budgets also override larger current CLI defaults.

7.2 Commands and separation of reproduction tasks

The following Bash commands run from the source repository root, with its working `.venv`. They document future reproduction and were not executed while writing this report. Select a currently idle single GPU UUID after inspecting `nvidia-smi`; do not assume the recorded card is still available.

Use fresh output roots to preserve existing successful results. Typical observed primary training times were about 3 minutes for CartPole and 54.48 minutes for MsPacman; MsPacman's final evaluation took about 4.19 minutes. These are hardware-specific observations, excluding environment setup and the wider validation suite.

```
# Reinspect nvidia-smi; use this recorded UUID only if still idle.
GPU_UUID=GPU-cef65388-88b0-f39a-6bac-c5316eb78540
export CUDA_VISIBLE_DEVICES="${GPU_UUID}"
export CUBLAS_WORKSPACE_CONFIG=:4096:8
export OMP_NUM_THREADS=1
export MPLBACKEND=Agg
CP=assignment2/results/reproduce_cartpole
MS=assignment2/results/reproduce_mspacman
.venv/bin/python -m assignment2.cartpole.train \
  --output "$CP/seed33" --seed 33 --episodes 1000 --max-steps 80000
.venv/bin/python -m assignment2.mspacman.train \
  --output "$MS/main_seed41" --seed 41 --episodes 4000 \
  --max-steps 2000000 --max-hours 4
```

For these exact measured runs, evaluation and plotting commands are below. A rerun can select a different checkpoint episode; read its `metrics/selection.json` and substitute the actual checkpoint filename. Both evaluators enforce exactly 500 episodes rather than taking a user-chosen episode count. Plotting uses saved curves and evaluation results, but also reloads the selected model to compute policy grids or image preferences.

```
.venv/bin/python -m assignment2.cartpole.evaluate \
  --checkpoint "$CP/seed33/checkpoints/best_ep0942.pt" \
  --output "$CP/seed33/evaluation500"
.venv/bin/python -m assignment2.cartpole.plots \
  --run "$CP/seed33" \
  --checkpoint "$CP/seed33/checkpoints/best_ep0942.pt" \
  --evaluation "$CP/seed33/evaluation500" --device cuda:0
.venv/bin/python -m assignment2.mspacman.evaluate \
  --checkpoint "$MS/main_seed41/checkpoints/best_ep03114.pt" \
  --output "$MS/main_seed41/evaluation500"
.venv/bin/python -m assignment2.mspacman.plotting \
  --run "$MS/main_seed41" --device cuda:0
```

The CartPole suite requires a completed seed-11 baseline and its finalization; it then runs the seven additional configurations sequentially. Its `finalize` command performs evaluation and plots, so do not also execute the individual evaluation command into the same directory. Diagnostics verify Bellman arithmetic and action selection; robustness changes only the initial-state distribution. MsPacman's short suite produces the six matched-cap runs and 30-reset diagnostics; the recovery smoke tests controlled interruption and rollback.

```
.venv/bin/python -m assignment2.cartpole.train \
  --output "$CP/main_seed11" --seed 11 --episodes 1000 --max-steps 80000
.venv/bin/python -m assignment2.cartpole.validate finalize --run "$CP/main_seed11"
.venv/bin/python -m assignment2.cartpole.validate suite --root "$CP" --max-steps 80000
.venv/bin/python -m assignment2.cartpole.validate diagnostics --run "$CP/seed33"
.venv/bin/python -m assignment2.cartpole.validate robustness --run "$CP/seed33"
.venv/bin/python -m assignment2.mspacman.validate short-suite \
  --root "$MS/validation" --steps 100000
.venv/bin/python -m assignment2.mspacman.validate diagnostics --run "$MS/main_seed41"
.venv/bin/python -m assignment2.mspacman.validate same-seed-prefix \
  --main "$MS/main_seed41" --short "$MS/validation/baseline_seed41"
```

```
.venv/bin/python -m assignment2.mspacman.smoke \
  --root "$MS/recovery_smoke" --rollback-test
CUDA_VISIBLE_DEVICES=-1 .venv/bin/python -m pytest assignment2/tests -q
```

For a complete fresh suite, let the suite create `seed33`; do not first run the separate seed-33 example into that same root. The examples are alternative routes, not a single script to concatenate. The source README gives individual ablation flags and full-resume commands. Figure-only MsPacman preference rendering from already measured Q-values is available with `assignment2.mspacman.plotting -run RUN -preferences-only`, requiring no new network inference. In contrast, compiling this report uses only the supplied PDF graphics, LaTeX sections, and bibliography: no Python, CSV input, checkpoint, GPU, shell escape, or external build step is required. No TeX compiler was available during preparation; compilation and page-layout verification remain to be performed in Overleaf, as documented in the report README.

References

[1] Marc G. Bellemare, Yavar Naddaf, Joel Veness, and Michael Bowling. The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research*, 47:253–279, 2013. https://arxiv.org/abs/1207.4708.

[2] Diego Klabjan. Deep Q-learning: DQN enhancements. MLDS 490 course lecture slides, Northwestern University. Supplied as Week 6 DQN.pdf, especially pp. 6–15 and 18–20.

[3] Marlos C. Machado, Marc G. Bellemare, Erik Talvitie, Joel Veness, Matthew Hausknecht, and Michael Bowling. Revisiting the arcade learning environment: Evaluation protocols and open problems for general agents. *Journal of Artificial Intelligence Research*, 61:523–562, 2018. https://arxiv.org/abs/1709.06009.

[4] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015. https://doi.org/10.1038/nature14236.

[5] Siqiao Mu. MLDS 490: Lab week 6. Northwestern University course lab handout, November 9, 2026. Supplied as MLDS_Lab_Week_6_1.pdf, especially pp. 4–10.

[6] Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. In *International Conference on Learning Representations*, 2016. https://arxiv.org/abs/1511.05952.

[7] Mark Towers, Ariel Kwiatkowski, Jordan Terry, et al. Gymnasium: A standard interface for reinforcement learning environments. arXiv:2407.17032, 2024. https://arxiv.org/abs/2407.17032.

[8] Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double Q-learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2016. https://arxiv.org/abs/1509.06461.

[9] Ziyu Wang, Tom Schaul, Matteo Hessel, Hado van Hasselt, Marc Lanctot, and Nando de Freitas. Dueling network architectures for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 1995–2003, 2016. https://proceedings.mlr.press/v48/wangf16.html.